

SKILLS & TECHNOLOGIES

RTL & Digital Design: Verilog, VHDL, FSM design, APB protocol, datapath design

EDA & Simulation Tools: QuestaSim, Xilinx Vivado, Synopsys PrimeTime (STA)

Programming & Scripting: Python, C/C++, Bash, Perl

Other Tools: Git, Linux, Docker, HPC/Slurm

EDUCATION

University of Victoria

Victoria, BC

Master of Applied Science in Electrical & Computer Engineering

Jan 2027 (Expected)

Laboratory Teaching Assistant: Microprocessor-Based Systems, Applied Electronics and Electrical Machines, Electronic Devices, Linear Circuits

Relevant Coursework: Digital Design, System-on-Chip Engineering for Signal Processing, Optimization for Machine Learning

Hanoi University of Science and Technology

Hanoi, Vietnam

Bachelor of Science in Electrical Engineering

Aug 2017 – Nov 2022

Relevant Coursework: Microcontroller Systems, Analog & Digital Electronics, Embedded Systems

WORK EXPERIENCE

University of Victoria

Graduate Research Assistant | Dr. Xiaodai Dong's Lab

Feb 2024 – Current

- Built and trained a hybrid CNN–RayBNN deep learning model in PyTorch and Rust for EEG-based drowsiness classification, running large-scale experiments on Compute Canada HPC clusters with Slurm-managed batch jobs.
- Implemented forward/backward propagation interfaces across a Python–Rust boundary, enabling end-to-end gradient flow between a PyTorch CNN and Rust-based classifier.

CoAsia SEMI

Digital Physical Design Engineer

Oct 2023 – Feb 2024

- Completed a structured training program on the ASIC backend flow, covering synthesis, floorplanning, place & route, clock tree synthesis, and timing closure through lectures, design reviews, and report analysis.
- Performed static timing analysis in Synopsys PrimeTime, debugging setup/hold violations, identifying critical paths, and applying SDC timing constraints via Tcl scripts to drive designs toward sign-off.
- Developed Tcl and shell scripts to parse PrimeTime timing reports, flag setup/hold violations, and summarize critical paths, streamlining the debug loop during iterative timing closure.

FPT Software

Software Engineer

May 2022 – May 2023

- Developed a C++ Matter Emulator on Linux and Raspberry Pi that simulated multiple Matter-compliant IoT devices (fans, thermostats, locks, lighting) on a single board, enabling automated protocol testing during product integration with LG Electronics.
- Wrote Bash automation scripts to auto-generate virtual IoT device clusters on Raspberry Pi 4, cutting device generation time by 80% and providing reproducible test environments for protocol testing.

PROJECTS

8-bit Programmable Timer IP with APB Slave Interface

- Designed an 8-bit timer IP in Verilog with three memory-mapped registers (TCR, TDR, TSR), up/down counting modes, 4 selectable internal clock sources, and overflow/underflow interrupt generation.
- Implemented an AMBA APB slave interface with FSM-controlled read/write logic and PSLVERR error response for invalid address access, ensuring full APB protocol compliance across 8-bit and 16-bit transfers.
- Built a Makefile-driven testbench in QuestaSim with a bus master VIP, clock/reset generators, and a scoreboard for self-checking, automating the full simulation and reporting flow via Tcl/Bash.
- Developed 21 directed, constrained-random, and concurrent (fork-join) test cases targeting register access, PSLVERR, up/down counting across 4 internal clocks, pause/resume, reset, and overflow/underflow corner cases — closing functional and code coverage holes.

FPGA-Based Real-Time Image Processing System

- Designed a real-time binary image processing pipeline in VHDL on a Nexys A7 FPGA, implementing morphological erosion, dilation, opening, and closing using a 3×3 structuring element.
- Implemented VGA synchronization and line-buffer timing logic to maintain 3×3 pixel-window alignment with display signals through the pipeline.

Hardware Accelerator Design in SystemC: Diffie-Hellman Key Exchange

- Designed a clocked hardware accelerator in SystemC to offload a compute-intensive digit-division function from the software side of a Diffie-Hellman key exchange, building a structural datapath of registers, multiplexers, adders, subtractors, and multipliers controlled by a Moore-type FSM.
- Replaced the default blocking FIFO communication between the hardware and software modules with a custom enable/done handshaking protocol, enabling cycle-level synchronization across the HW/SW boundary.